

Docker 容器应用



主题	Docker 容器应用
文档号	1.0
创建时间	2025-05-08
最后修改	2025-05-08
版本号	1.0
文件名	Docker 容器应用 V1. 0.pdf
文件格式	Portable Document Format

阅前须知

声明

北京合众恒跃科技有限公司保留随时对其产品进行修正、改进和完善的权利，客户在下单前应获取相关信息的最新版本，并验证这些信息是正确的。本文档一切解释权归北京合众恒跃科技有限公司所有。

简介

北京合众恒跃科技有限公司位于中关村科技园区，公司主要从事嵌入式产品的设计、研发、生产、销售等业务；公司的核心竞争力是对嵌入式产品精准的设计及实现能力，成本控制能力和产品品质保障能力；公司主干研发人员均有多年嵌入式产品开发经验和软硬件技术积累，服务于音视频、图像识别、电力等多个应用领域，公司研发设计的视频转码卡、图像识别卡、电力保护装置等产品在业内享有一定的知名度。

联系方式

北京合众恒跃科技有限公司

电话：010-62129511

邮编：102208

技术支持邮箱：support@hzhytech.com

地址：北京市海淀区安宁庄后街南 1 号 A 区一层 1020 号



官方微信公众号



官方企业微信



合众嵌入式官方店铺



合众 AI 官方店铺

修改记录

版本号	日期	修改人	备注
1.0	2025-05-08	李园园、米泓宇	初始版本

目录

阅前须知	2
声明	2
简介	2
联系方式	2
修改记录	3
目录	4
第 1 章 简介	6
1.1 Docker 简介	6
1.2 Docker 在嵌入式设备中的优势	6
1.3 文档适用范围	6
第 2 章 环境准备	8
2.1 检查 Docker 安装状态	8
2.2 嵌入式开发板 Docker 环境说明	8
第 3 章 Docker 基础概念	9
3.1 镜像 (Image)	9
3.2 容器 (Container)	9
3.3 相关术语简述	9
第 4 章 基础操作演示	10
4.1 镜像管理	10
4.1.1 查看本地镜像 (docker images)	10
4.1.2 删除镜像 (docker rmi)	10
4.2 容器管理	10
4.2.1 运行容器 (基于本地镜像)	10
4.2.2 查看运行中的容器 (docker ps)	11

4.2.3 进入容器 (docker exec)	11
4.2.4 停止容器 (docker stop)	11
4.2.5 删除容器 (docker rm)	12
4.2.6 查看容器日志 (docker logs)	12
第 5 章 容器移植与跨平台部署	13
5.1 镜像导出与导入	13
5.1.1 导出镜像 (docker save)	13
5.1.2 导入镜像 (docker load)	13
5.2 检查 Docker Buildx 环境	14
5.2.1 安装 qemu	14
5.2.2 注册 qemu 到 docker	14
5.2.3 检查 buildx 是否安装	14
5.2.4 自动安装 buildx	15
5.2.5 手动安装 buildx	15
5.3 Docker Buildx 多平台构建	15
5.3.1 初始化 buildx 多平台构建器	15
5.3.2 构建多平台镜像	16
第 6 章 实用示例	19
6.1 基于本地 Ubuntu 镜像定制新镜像	19
6.1.1 运行 Ubuntu 容器并安装软件	19
6.1.2 提交容器为新镜像	19
6.1.3 使用新镜像启动容器	20

第 1 章 简介

1.1 Docker 简介

Docker 是一种开源的容器化平台，允许开发者将应用程序及其所有依赖打包到一个标准化、轻量级的容器中。容器能够在任何支持 Docker 的环境中快速启动和运行，保证应用行为一致。

相比传统虚拟机，Docker 容器启动更快，占用资源更少，适合现代软件开发和部署需求。

Docker 容器技术基于 Linux 内核的命名空间（namespace）和控制组（cgroups）实现资源隔离与限制，使得容器既轻量又安全。

1.2 Docker 在嵌入式设备中的优势

- 轻量级部署

容器相比虚拟机更节省资源，适合资源有限的嵌入式设备。

- 环境一致性

容器内包含应用及其依赖，避免“环境不一致”导致的问题。

- 快速迭代与升级

通过镜像快速更新应用版本，简化升级流程。

- 统一管理

Docker 提供标准化命令行工具，便于应用管理与维护。

- 便于移植和扩展

容器镜像可在不同硬件平台和系统间迁移，方便公司多款开发板统一管理。

1.3 文档适用范围

本文档适用于所有已安装 Docker 的设备，包括 Ubuntu 20.04 桌面环境和各种嵌入式开发板。

所有演示均基于 Ubuntu 20.04 系统进行，嵌入式开发板操作步骤完全一致，方便跨平台使用和技能迁移。

第 2 章 环境准备

2.1 检查 Docker 安装状态

打开终端，输入以下命令检查 Docker 版本：

```
docker --version
```

```
root@ubuntu:~# docker --version
Docker version 24.0.7, build 24.0.7-0ubuntu2~20.04.1
```

图 2.1-1 查看 Docker 版本

如果提示“command not found”或无版本信息，说明 Docker 未安装，如有需要请自行安装。

2.2 嵌入式开发板 Docker 环境说明

在嵌入式开发板上，如果已经部署了 Docker，以上检查步骤同样适用。可以通过执行 `docker --version` 确认 Docker 版本，通过 `docker run hello-world` 验证 Docker 环境是否正常。

注意：所有操作命令在 Ubuntu 20.04 和嵌入式设备上完全一致，方便开发者跨平台使用。

第 3 章 Docker 基础概念

3.1 镜像 (Image)

镜像是一个只读的模板，包含运行应用所需的代码、库、环境变量和配置，是容器的基础。镜像可以从公共仓库（如 Docker Hub）拉取，也可以自定义构建。

3.2 容器 (Container)

容器是镜像的一个可运行实例。它是一个独立的、可执行的软件包，包含应用及其所有依赖。容器启动后，可以像普通进程一样运行、停止、删除。

3.3 相关术语简述

- 仓库 (Repository)：存储镜像的地方，常用的公共仓库是 Docker Hub。
- Docker Daemon：后台服务，负责管理容器和镜像。
- Docker Client：用户通过命令行与 Docker 交互的工具。
- Dockerfile：用于定义镜像构建步骤的文本文件。
- 数据卷 (Volume)：用于持久化存储容器数据的机制。

第 4 章 基础操作演示

4.1 镜像管理

4.1.1 查看本地镜像（docker images）

查看当前系统中已有的镜像列表：

```
docker images
```

```
root@ubuntu:~# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED
ubuntu              latest             ba6acccedd29       3 years ago
training/webapp     latest             6fae60ef3446       9 years ago
```

图 4.1-1 查看镜像

4.1.2 删除镜像（docker rmi）

如果需要删除某个镜像，先停止并删除相关容器，然后执行：

```
docker rmi 镜像名或镜像 ID
```

例如删除 training/webapp 镜像：

```
docker rmi training/webapp:latest
```

4.2 容器管理

4.2.1 运行容器（基于本地镜像）

基于本地已有的 Ubuntu 镜像启动一个交互式容器：

```
docker run -it --name my_ubuntu_container ubuntu:latest /bin/bash
```

```
root@ubuntu:~# docker run -it --name my_ubuntu_container ubuntu:latest /bin/bash
root@c9eb663b3ed3:/#
root@c9eb663b3ed3:/# ls
bin  dev  home  lib32  libx32  mnt  proc  run  srv  tmp  var
boot  etc  lib  lib64  media  opt  root /sbin  sys  usr
root@c9eb663b3ed3:/# uname -a
Linux c9eb663b3ed3 4.15.0-142-generic #146~16.04.1-Ubuntu SMP Tue Apr 13 09:27:15 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
```

图 4.2-1 启动容器

基于本地已有的 training/webapp 镜像启动容器（假设该镜像包含可执行应用）：

```
docker run -d --name my_webapp_container -p 8080:80 training/webapp:latest
```

```
root@ubuntu:~# docker run -d --name my_webapp_container -p 8080:80 training/webapp:latest
01b8b8f41e22f485296e59375a8951a1b1fa88f88e550ea26e75d3ebe86bea0a
```

图 4.2-2 启动容器

这里-d 表示后台运行，-p 映射端口。

4.2.2 查看运行中的容器（docker ps）

查看当前正在运行的容器：

```
docker ps
```

```
root@ubuntu:~# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS
PORTS         NAMES
01b8b8f41e22   training/webapp:latest "python app.py"         2 minutes ago Up 2 minutes
5000/tcp, 0.0.0.0:8080->80/tcp my_webapp_container
c9eb663b3ed3   ubuntu:latest       "/bin/bash"             3 minutes ago Up 3 seconds
my_ubuntu_container
```

图 4.2-3 查看运行容器

4.2.3 进入容器（docker exec）

进入正在运行的 Ubuntu 容器交互操作：

```
docker exec -it my_ubuntu_container /bin/bash
```

```
root@ubuntu:~# docker exec -it my_ubuntu_container /bin/bash
root@c9eb663b3ed3:/# ls
bin  dev  home  lib32  libx32  mnt  proc  run  srv  tmp  var
boot  etc  lib  lib64  media  opt  root /sbin  sys  usr
```

图 4.2-4 进入容器

4.2.4 停止容器（docker stop）

停止指定容器：

```
docker stop my_ubuntu_container
```

```
root@ubuntu:~# docker stop my_ubuntu_container
my_ubuntu_container
root@ubuntu:~# docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS
01b8b8f41e22	training/webapp:latest	my_webapp_container	"python app.py"	4 minutes ago	Up 4 minutes

```
root@ubuntu:~#
```

图 4.2-5 停止容器

```
docker stop my_webapp_container
```

```
root@ubuntu:~# docker stop my_webapp_container
my_webapp_container
root@ubuntu:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PO
RTS					

图 4.2-6 停止容器

4.2.5 删除容器 (docker rm)

删除已停止的容器:

```
docker rm my_ubuntu_container
docker rm my_webapp_container
```

4.2.6 查看容器日志 (docker logs)

查看 ubuntu 容器日志:

```
docker logs my_ubuntu_container
```

```
root@ubuntu:~# docker logs my_ubuntu_container
root@c9eb663b3ed3:/#
root@c9eb663b3ed3:/# ls
bin  dev  home  lib32  libx32  mnt  proc  run  srv  tmp  var
boot  etc  lib  lib64  media  opt  root  sbin  sys  usr
root@c9eb663b3ed3:/# uname -a
Linux c9eb663b3ed3 4.15.0-142-generic #146~16.04.1-Ubuntu SMP Tue Apr 13 09:27:15 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
root@c9eb663b3ed3:/# exit
exit
root@c9eb663b3ed3:/# exit
```

图 4.2-7 查看容器日志

第 5 章 容器移植与跨平台部署

5.1 镜像导出与导入

5.1.1 导出镜像（docker save）

在源设备导出镜像：

```
docker save -o my_image.tar image_name:tag
```

```
hzhy@ubuntu:~/docker$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world    latest    74cc54e27dc4   3 months ago   10.1kB
hzhy@ubuntu:~/docker$
hzhy@ubuntu:~/docker$ sudo docker save -o hello-world.tar hello-world:latest
hzhy@ubuntu:~/docker$ ls
hello-world.tar
hzhy@ubuntu:~/docker$
```

图 5.1-1 导出 Docker 镜像

5.1.2 导入镜像（docker load）

将导出的镜像包拷贝到目标系统后，导入镜像：

```
docker load -i my_image.tar
```

```
hzhy@ubuntu:~$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hzhy@ubuntu:~$
hzhy@ubuntu:~$ sudo docker load -i hello-world.tar
63a41026379f: Loading layer [=====]
Loaded image: hello-world:latest
hzhy@ubuntu:~$
hzhy@ubuntu:~$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world    latest    74cc54e27dc4   3 months ago   10.1kB
hzhy@ubuntu:~$
```

图 5.1-2 导入 Docker 镜像


```
hzhy@ubuntu:~$ docker buildx version
github.com/docker/buildx v0.22.0 18ccba0
```

图 5.2-3 docker buildx

若输出版本信息，则 buildx 已安装，可跳过 5.2.2、5.2.3 小节；若提示命令不存在，需安装 buildx。

5.2.4 自动安装 buildx

参考 Docker 官方文档 [Docker 官方文档-Ubuntu 安装 Docker 及插件](#)。

5.2.5 手动安装 buildx

确认 buildx 是否已存在但未启用：

```
ls -l /usr/libexec/docker/cli-plugins/docker-buildx
```

若文件存在，直接启用：

```
export DOCKER_CLI_EXPERIMENTAL=enabled
docker buildx install
```

若文件不存在，手动下载并安装：

```
mkdir -p /usr/libexec/docker/cli-plugins
wget -O /usr/libexec/docker/cli-plugins/docker-buildx \
https://github.com/docker/buildx/releases/download/v0.22.0/buildx-v0.22.0.linux-amd64
chmod +x /usr/libexec/docker/cli-plugins/docker-buildx
```

验证 buildx 是否可用：

```
docker buildx version
```

```
hzhy@ubuntu:~$ docker buildx version
github.com/docker/buildx v0.22.0 18ccba0
```

图 5.2-4 docker buildx

5.3 Docker Buildx 多平台构建

5.3.1 初始化 buildx 多平台构建器

创建构建器：

```
docker buildx create --name cross-builder --use
```

```
hzhy@ubuntu:~/docker$ sudo docker buildx create --name cross-builder --use cross-builder
hzhy@ubuntu:~/docker$
```

图 5.3-1 创建构建器

验证构建器状态:

```
docker buildx inspect --bootstrap
```

```
hzhy@ubuntu:~/docker$ sudo docker buildx inspect --bootstrap
[+] Building 2.9s (1/1) FINISHED
=> [internal] booting buildkit
=> => pulling image moby/buildkit:buildx-stable-1
=> => creating container buildx_buildkit_cross-builder0
Name:          cross-builder
Driver:        docker-container
Last Activity: 2025-05-09 06:38:23 +0000 UTC

Nodes:
Name:          cross-builder0
Endpoint:      unix:///var/run/docker.sock
Status:        running
BuildKit daemon flags: --allow-insecure-entitlement=network.host
BuildKit version: v0.21.1
Platforms:     linux/amd64, linux/amd64/v2, linux/amd64/v3, linux/amd64/v4, li
4, linux/ppc64le, linux/s390x, linux/386, linux/mips64le, linux/mips64, linux/loong64,
```

图 5.3-2 验证构建器

查看可用构建器:

```
docker buildx ls
```

```
hzhy@ubuntu:~/docker$ sudo docker buildx ls
NAME/NODE           DRIVER/ENDPOINT   STATUS   BUILDKIT     PLATFORMS
cross-builder*      docker-container  running  v0.21.1      linux/amd64 (+4),
/arm64, linux/arm (+2), linux/ppc64le, (7 more)
default             docker           running  v0.11.7+d3e6c1360f6e  linux/amd64 (+4),
/arm64, linux/arm (+2), linux/ppc64le, (6 more)
hzhy@ubuntu:~/docker$
```

图 5.3-3 查看可用构建器

5.3.2 构建多平台镜像

切换到本地默认构建器:

```
docker buildx use default
```



```
hzhy@ubuntu:~/docker$ sudo docker buildx use default
hzhy@ubuntu:~/docker$
hzhy@ubuntu:~/docker$ sudo docker buildx ls
NAME/NODE                DRIVER/ENDPOINT          STATUS
cross-builder            docker-container
  \_ cross-builder0      \_ unix:///var/run/docker.sock  running
(+2), linux/ppc64le, (7 more)
default*                  docker
  \_ default              \_ default                    running
(+2), linux/ppc64le, (6 more)
hzhy@ubuntu:~/docker$
```

图 5.3-4 切换本地默认构建器

手动拉取镜像或者使用资料包中提供的镜像：

```
docker pull --platform linux/arm64 alpine:latest
```

```
hzhy@ubuntu:~/docker$ sudo docker pull --platform linux/arm64 alpine:latest
latest: Pulling from library/alpine
6e771e15690e: Already exists
Digest: sha256:a8560b36e8b8210634f77d9f7f9efd7ffa463e380b75e2e74aff4511df3ef88c
Status: Downloaded newer image for alpine:latest
docker.io/library/alpine:latest
hzhy@ubuntu:~/docker$
hzhy@ubuntu:~/docker$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
alpine        latest    8d591b0b7dea   2 months ago   8.17MB
hzhy@ubuntu:~/docker$
```

图 5.3-5 创建简单镜像

构建 ARM 平台镜像：

```
sudo docker buildx build --platform linux/arm64 -t my-arm-image:latest --load .
```

```
hzhy@ubuntu:~/docker$ sudo docker buildx build --platform linux/arm64 -t my-arm-image:latest --load .
[+] Building 0.0s (5/5) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 83B
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load metadata for docker.io/library/alpine:latest
=> CACHED [1/1] FROM docker.io/library/alpine:latest
=> exporting to image
=> => exporting layers
=> => writing image sha256:a4fcad98452d4ce470dec5a461a3700eebd557137a145154de4d1a203f4c8f2d
=> => naming to docker.io/library/my-arm-image:latest
hzhy@ubuntu:~/docker$
```

图 5.3-6 构建 ARM 镜像

```
hzhy@ubuntu:~/docker$ sudo docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
my-arm-image        latest             a4fcad98452d       2 months ago       8.17MB
alpine              latest             8d591b0b7dea       2 months ago       8.17MB
hzhy@ubuntu:~/docker$
```

图 5.3-7 查看构建的镜像

验证镜像架构：

```
docker inspect my-arm-image | grep Architecture
```

```
hzhy@ubuntu:~/docker$
hzhy@ubuntu:~/docker$ sudo docker inspect my-arm-image | grep Architecture
    "Architecture": "arm64",
hzhy@ubuntu:~/docker$
hzhy@ubuntu:~/docker$
```

图 5.3-8 验证镜像架构

第 6 章 实用示例

6.1 基于本地 Ubuntu 镜像定制新镜像

6.1.1 运行 Ubuntu 容器并安装软件

启动容器：

```
docker run -it --name temp_ubuntu ubuntu:latest /bin/bash
```

```
root@ubuntu:~# docker run -it --name temp_ubuntu ubuntu:latest /bin/bash
root@9e2bb86c1613:/# ls
bin  dev  home  lib32  libx32  mnt  proc  run  srv  tmp  var
boot  etc  lib  lib64  media  opt  root  sbin  sys  usr
root@9e2bb86c1613:/#
```

图 6.1-1 启动容器

容器内执行：

```
root@9e2bb86c1613:/# touch make_image_test
root@9e2bb86c1613:/# echo "hello world" >make_image_test
root@9e2bb86c1613:/#
```

图 6.1-2 安装软件

安装完成后，输入 `exit` 退出容器。

```
root@9e2bb86c1613:/# exit
exit
root@ubuntu:~#
```

图 6.1-3 安装完成

6.1.2 提交容器为新镜像

将修改后的容器保存为新镜像：

```
docker commit temp_ubuntu my_custom_ubuntu:1.0
```

```
root@ubuntu:~# docker commit temp_ubuntu my_custom_ubuntu:1.0
sha256:436706c28c83a6ca990f275fd7e6eef8c48103cd0979d65773b0e9bb8b7dd9a7
root@ubuntu:~#
```

图 6.1-4 保存容器为新镜像

6.1.3 使用新镜像启动容器

```
docker run -it --name custom_ubuntu_container my_custom_ubuntu:1.0 /bin/bash
```

```
root@ubuntu:~# docker commit temp_ubuntu my_custom_ubuntu:1.0
sha256:436706c28c83a6ca990f275fd7e6eef8c48103cd0979d65773b0e9bb8b7dd9a7
root@ubuntu:~# docker run -it --name custom_ubuntu_container my_custom_ubuntu:1.0 /bin/bash
root@152a69dff3ce:/# ls
bin    dev    home  lib32  libx32      media  opt    root  sbin  sys  usr
boot  etc    lib   lib64  make_image_test mnt    proc   run   srv   tmp  var
root@152a69dff3ce:/# cat make_image_test
hello world
```

图 6.1-5 启动容器